AIBioBench

Pass 1 - Easy SQL

Kris Kari | https://github.com/karikris/AIBioBench

Introduction

Pass 1 of AIBioBench contains eight foundational SQL-style tests designed to probe whether a model can handle the basics of structured data reasoning with exactness. Across these eight cases, the suite checks simple and chained joins, row preservation versus row removal, sort order obedience, and grouped summaries such as counts and averages, with the goal of discovering whether a model can behave like a reliable query engine on small tables before moving on to harder passes with more complex analytical logic.

The tasks looked simple, but the dataset contained relational traps such as orphan sample keys, orphan variant keys, and a zero call dimension row that still had to be preserved in the right query types.

The main traps were:

- Orphan foreign keys: S999 exists in fact_calls but not sample_dim.
- Orphan variant: V999 exists in fact_calls but not variant_dim.
- Zero-call dimension row: S5 exists in sample_dim but has no matching calls.
- Second inner join trap: V5 -> G999, but G999 is absent from gene_dim.
- Aggregate weighting: averages and counts must be over joined fact rows, not over distinct samples, variants, or plausible biological categories.

Standard SQL instructions

You are given four CSV tables in a snowflake schema:

```
fact_calls
 ├─── sample_dim
 └─── variant_dim
     └─── gene_dim
```

Rules:
1. Use ANSI SQL.
2. Show the SQL first, then show the final result table.
3. Use NULL for missing values.
4. Preserve the exact requested sort order.
5. Round derived decimal values to 3 decimals.
6. If your SQL dialect does not support RIGHT JOIN, use an equivalent reversed LEFT JOIN.
7. If your SQL dialect does not support FULL OUTER JOIN, emulate it correctly.
8. Do not invent rows or columns.
9. Unless explicitly stated otherwise, summaries that follow a join should be computed over the joined rows, not over distinct source rows.

Prompt 1: Basic relational join accuracy

Overview

This test measures if a model can read a simple relational instruction and translate it into correct table operation. Checks several core skills at once. A model must identify fact_calls as the driving table, match it to sample_dim using sample_id, select the exact requested columns, and preserve the required sort order by call_id ascending. A model must understand join semantics, avoid inventing extra columns, keep column order exact, and return results in a deterministic order. A model that gets this right shows baseline schema literacy. Weak models often make one of four mistakes: they use the wrong join type, they pull condition or tissue from the wrong place, they change the requested column order, or they forget the explicit sort requirement.

Walkthrough

Start from fact_calls as f.
Join sample_dim as s on f.sample_id = s.sample_id.
Select exactly f.call_id, f.sample_id, s.tissue, s.condition, and f.genotype.
Then order final result by f.call_id ASC

Discovery

Both Gemma 31B models were exact. Most models leaked S999 into the result as if the join were left-join-like. The Gemma 26B variants went the other way and dropped call_id 9 apparently to over-apply the variant join constraint even if Q1 only joined samples.

Pass 1 - Easy SQL | Prompt 1

Prompt

Inner join fact_calls to sample_dim on sample_id.
Return: call_id, sample_id, tissue, condition, genotype
Sort by call_id ascending.

Code

```
SELECT
    f.call_id,
    f.sample_id,
    s.tissue,
    s.condition,
    f.genotype
FROM fact_calls AS f
INNER JOIN sample_dim AS s
  ON f.sample_id = s.sample_id
ORDER BY f.call_id ASC;
```

Result

call_id	sample_id	tissue	condition	genotype
1	S1	mature_leaf	high_light	0/1
2	S1	mature_leaf	high_light	1/1
3	S2	mature_leaf	control	0/0
4	S2	mature_leaf	control	0/1
5	S3	young_leaf	drought	0/1
6	S3	young_leaf	drought	1/1
7	S4	young_leaf	control	0/0
9	S2	mature_leaf	control	0/1

Prompt 2: Variant join and ranking accuracy

Overview

This test measures whether a model can correctly join fact_calls to variant_dim using variant_id, retrieve attributes from the right table, and apply a two level sort rule exactly as written. The model must return call_id, variant_id, variant_class, impact, and qual, with qual sorted descending and ties broken by call_id ascending. A model has to understand that variant_class and impact come from the dimension table while qual remains in the fact table. A model that gets this prompt right shows that it can track table roles, map columns to the correct source, preserve the requested output schema, and execute deterministic ranking logic. Weak models often use the wrong join key, confuse fact table fields with dimension table fields, sort qual in ascending order by mistake, or forget the tie break on call_id. Some models also return extra columns or rearrange the requested column order, which breaks automated scoring.

Walkthrough

Begin with fact_calls as f.
Join variant_dim as v on f.variant_id = v.variant_id.
Select exactly f.call_id, f.variant_id, v.variant_class, v.impact, and f.qual.
Then sort the final result with the highest qual first and, where qual values are equal, place the smaller call_id first.

Pass 1 - Easy SQL | Prompt 2

Prompt

Inner join fact_calls to variant_dim on variant_id.
Return: call_id, variant_id, variant_class, impact, qual
Sort by qual descending, then call_id ascending.

Code

```
SELECT
    f.call_id,
    f.variant_id,
    v.variant_class,
    v.impact,
    f.qual
FROM fact_calls AS f
INNER JOIN variant_dim AS v
  ON f.variant_id = v.variant_id
ORDER BY f.qual DESC, f.call_id
ASC;
```

Result

call_id	variant_id	variant_class	impact	qual
1	V1	SNV	moderate	99
3	V1	SNV	moderate	95
6	V5	SNV	modifier	92
7	V3	SNV	high	88
2	V2	indel	high	87
4	V3	SNV	high	80
5	V4	SNV	low	70
8	V2	indel	high	60

Prompt 3: Left join row preservation and nullable enrichment

Overview

This test measures whether a model understands LEFT JOIN behaviour. A model must keep every row from fact_calls, attach matching values from sample_dim when they exist, and return NULL values for tissue, batch, or expr_ndhb when there is no matching sample row. A model that gets this right shows that it understands row preservation, table roles, missing data behaviour and discipline in returning exact requested columns in the exact requested order. Weak models often replace the left join with an inner join, which incorrectly drops unmatched fact rows. Other common mistakes are pulling the wrong columns, forgetting that unmatched dimension values should remain null, or ignoring the sort rule. If a model performs well on this prompt, it suggests that it can handle incomplete relational links without silently losing evidence. That is a very important sign for real analytical work, because preserving unmatched records is often essential for quality control, auditing, and trustworthy downstream summaries.

Walkthrough

Start from fact_calls as the base table.
Use LEFT JOIN to connect sample_dim on sample_id.
Select exactly f.call_id, f.sample_id, s.tissue, s.batch, and s.expr_ndhb.
Do not filter out rows with missing sample matches.
Then sort the final result by f.call_id in ascending order.

Discovery

This was the only broadly solved task, with 7/9 exact. It failed mainly for Phi4 Mini, which misplaced or invented values, and Qwen Coder 30B 256k, which returned string call_id values instead of integers.

Pass 1 - Easy SQL | Prompt 3

Prompt

Inner join fact_calls to sample_dim on sample_id.
Return: call_id, sample_id, tissue, condition, genotype
Sort by call_id ascending.

Code

```
SELECT
    f.call_id,
    f.sample_id,
    s.tissue,
    s.batch,
    s.expr_ndhb
FROM fact_calls AS f
LEFT JOIN sample_dim AS s
  ON f.sample_id = s.sample_id
ORDER BY f.call_id ASC;
```

Result

call_id	sample_id	tissue	batch	expr_ndhb
1	S1	mature_leaf	B1	1200
2	S1	mature_leaf	B1	1200
3	S2	mature_leaf	B1	300
4	S2	mature_leaf	B1	300
5	S3	young_leaf	B2	900
6	S3	young_leaf	B2	900
7	S4	young_leaf	B2	100
8	S999	NULL	NULL	NULL
9	S2	mature_leaf	B1	300

Prompt 4: Sample preserving aggregation with zero call coverage

Overview

This test measures whether a model understands sample preserving joins, grouped aggregation, and missing match behaviour. A model must return one row for every sample in sample_dim, even when a sample has no matching rows in fact_calls. It also must compute call_count and avg_qual correctly at the sample level. A model must choose the correct join direction, preserve dimension rows with zero matches, and aggregate only after the join is set up correctly. Common errors include using an inner join, which drops samples with zero calls, grouping at the wrong level, counting rows incorrectly, or rounding too early. Another frequent mistake is forgetting that the prompt wants one row per sample, not one row per fact call. Tests whether a model can preserve the analytical frame of reference. If a model performs well on this prompt, it suggests that it can handle coverage style reporting, preserve zero count groups, and produce trustworthy summaries over incomplete event data.

Walkthrough

Start from sample_dim because the output must include every sample.
Join fact_calls to it with a LEFT JOIN, which is the same idea as reversing the prompt's RIGHT JOIN.
Group by s.sample_id and s.tissue.
Count f.call_id to get the number of matched calls for each sample.
Average f.qual for matched rows only, cast for precise rounding, and round final average to 3 decimals.
Then sort by s.sample_id ascending.

Discovery

4/9 were exact. The models mostly understood that zero-call samples should appear, but several miscomputed the S2 aggregate. Gold for S2 is call_count=3, avg_qual=76.667; several outputs used two calls or the wrong average.

Pass 1 - Easy SQL | Prompt 4

Prompt

Use a RIGHT JOIN from fact_calls to sample_dim, or an equivalent reversed LEFT JOIN.
Return one row per sample with:
sample_id, tissue, call_count, avg_qual
Include samples with zero calls.
Sort by sample_id ascending.
Round avg_qual to 3 decimals.

Code

```
SELECT
    s.sample_id,
    s.tissue,
    COUNT(f.call_id) AS call_count,
    ROUND(AVG(CAST(f.qual AS
    DECIMAL(18, 6))), 3) AS avg_qual
FROM sample_dim AS s
LEFT JOIN fact_calls AS f
  ON s.sample_id = f.sample_id
GROUP BY
    s.sample_id,
    s.tissue
ORDER BY s.sample_id ASC;
```

Result

sample_id	tissue	call_count	avg_qual
S1	mature_leaf	2	93.000
S2	mature_leaf	3	76.667
S3	young_leaf	2	81.000
S4	young_leaf	1	88.000
S5	root	0	NULL

Prompt 5: Multi table join through variant to gene

Overview

This prompt measures whether a model can follow a join chain across three related tables instead of only matching one fact table to one dimension table. Many models can handle a single join, but a chained join is harder because the model must understand that gene_symbol and pathway are not directly available from fact_calls. It has to move through variant_dim first. A model that gets this prompt right shows that it understands relational structure, foreign key style linking, and column provenance. It knows which table provides each field and can build the correct path through the schema without inventing shortcuts or skipping required links. Weak models often try to join fact_calls directly to gene_dim, which is incorrect. Others may use the wrong join key, forget one of the joins, or return columns in the wrong order. If a model performs well on this prompt, it suggests that it can follow schema relationships accurately across multiple tables and preserve exact output structure.

Walkthrough

Begin with fact_calls as f.
Join variant_dim as v on f.variant_id = v.variant_id.
Then join gene_dim as g on v.gene_id = g.gene_id.
Select exactly f.call_id, f.variant_id, g.gene_symbol, and g.pathway.
Finally, order the results by f.call_id ASC.

Pass 1 - Easy SQL | Prompt 5

Prompt

Inner join fact_calls to variant_dim, then to gene_dim.
Return: call_id, variant_id, gene_symbol, pathway
Sort by call_id ascending.

Code

```
SELECT
    f.call_id,
    f.variant_id,
    g.gene_symbol,
    g.pathway
FROM fact_calls AS f
INNER JOIN variant_dim AS v
  ON f.variant_id = v.variant_id
INNER JOIN gene_dim AS g
  ON v.gene_id = g.gene_id
ORDER BY f.call_id ASC;
```

Result

call_id	variant_id	gene_symbol	pathway
1	V1	NDHB	chloroplast_NDH_complex
2	V2	NDHB	chloroplast_NDH_complex
3	V1	NDHB	chloroplast_NDH_complex
4	V3	NDHK	chloroplast_NDH_complex
5	V4	PGR5	cyclic_electron_flow
7	V3	NDHK	chloroplast_NDH_complex
8	V2	NDHB	chloroplast_NDH_complex

Prompt 6: Grouped aggregation by impact

Overview

This prompt measures whether a model can combine an inner join with grouped aggregation. The model must join fact_calls to variant_dim on variant_id, group the joined rows by impact, count how many calls fall into each impact class, and compute the average quality score for each group. A model has to understand where the grouping field comes from, where the numeric field comes from, and how to aggregate correctly after the join. A model that gets this right shows that it can track column provenance across tables, choose the correct grouping level, and compute multiple summary statistics in one result. Weak models often group by the wrong field, forget the join, average the wrong column, or sort alphabetically instead of by count first. If a model performs well on this prompt, it suggests that it can handle one of the most important patterns in analytics: join, group, summarise, and rank.

Walkthrough

Start from fact_calls as f.
Join variant_dim as v on f.variant_id = v.variant_id.
Group the joined rows by v.impact.
Count all rows in each group to produce call_count.
Average f.qual within each group and round the result to 3 decimals for avg_qual.
Finally, sort the result by call_count descending and impact ascending.

Discovery

0/9 exact. Models undercounted duplicate high-impact calls and produced wrong averages. They often treated variants as unique entities instead of counting every joined fact row.

Pass 1 - Easy SQL | Prompt 6

Code

```
SELECT
    v.impact,
    COUNT(*) AS call_count,
    ROUND(AVG(CAST(f.qual AS
    DECIMAL(18, 6))), 3) AS avg_qual
FROM fact_calls AS f
INNER JOIN variant_dim AS v
  ON f.variant_id = v.variant_id
GROUP BY v.impact
ORDER BY call_count DESC, v.impact ASC;
```

Prompt

Inner join fact_calls to variant_dim.
Count calls by impact and also compute average qual.
Return: impact, call_count, avg_qual
Sort by call_count descending, then impact ascending.
Round avg_qual to 3 decimals.

Result

impact	call_count	avg_qual
high	4	78.750
moderate	2	97.000
low	1	70.000
modifier	1	92.000

Prompt 7: Tissue level quality summary

Overview

This prompt measures whether a model can combine an inner join with grouped numerical aggregation. The model must join fact_calls to sample_dim on sample_id, group the joined rows by tissue, and compute three summary statistics from qual: the average, the minimum, and the maximum. The model has to identify the correct grouping column from sample_dim, apply multiple aggregate functions to the qual field from fact_calls, and follow a precise output contract. Many models can do a simple count, but fewer models can keep column provenance, aggregation logic, rounding, and sorting all correct at the same time. A model that gets this right shows that it understands how to summarise fact data by a dimension attribute. Weak models often group by the wrong field, forget the join, apply rounding to all numeric columns instead of only avg_qual, or sort by tissue instead of by the computed average. Another common mistake is to treat avg_qual as a raw column instead of a derived summary.

Walkthrough

Start with fact_calls as f and join sample_dim as s on f.sample_id = s.sample_id.
Group the joined data by s.tissue.
Compute AVG(f.qual) for avg_qual, MIN(f.qual) for min_qual, and MAX(f.qual) for max_qual.
Round only the average to 3 decimals.
Return the columns in this exact order: tissue, avg_qual, min_qual, max_qual.
Then sort the final result by avg_qual in descending order.

Discovery

0/9 exact and the worst mean aligned cell accuracy. The expected result is weighted by fact-call rows after joining samples. Models instead behaved as if aggregating at the sample level, included root/S5, dropped valid repeated calls, or used the wrong denominator.

Pass 1 - Easy SQL | Prompt 7

Code

```
SELECT
    s.tissue,
    ROUND(AVG(CAST(f.qual  AS  DECIMAL(18,
6))), 3) AS avg_qual,
    MIN(f.qual) AS min_qual,
    MAX(f.qual) AS max_qual
FROM fact_calls AS f
INNER JOIN sample_dim AS s
    ON f.sample_id = s.sample_id
GROUP BY s.tissue
ORDER BY avg_qual DESC;
```

Prompt

Inner join fact_calls to sample_dim.
Compute average, minimum, and maximum qual by tissue.
Return: tissue, avg_qual, min_qual, max_qual
Sort by avg_qual descending.
Round avg_qual to 3 decimals.

Result

tissue	avg_qual	min_qual	max_qual
young_leaf	83.333	70	92
mature_leaf	83.200	55	99

Prompt 8: Condition level call volume and joined expression summary

Overview

This prompt measures whether a model can join fact_calls to sample_dim, group the joined rows by condition, and compute two different summaries at the same grouping level. A model must count joined call rows and also average expr_pgr5 from the sample table across those joined rows. Checks whether the model understands analytical grain and exposes a very common reasoning failure in AI systems: confusing row-level weighting with entity-level weighting. Since the join can duplicate sample attributes across multiple fact rows, the model must understand that the requested average is over the joined dataset, not over distinct samples. It also shows that it can follow exact formatting rules such as column order, alphabetical sorting by condition, and rounding avg_expr_pgr5 to 3 decimals. If a model performs well on this prompt, it suggests that it can reason correctly about grouping grain, join induced duplication, and mixed aggregation over fact and dimension fields.

Walkthrough

Start from fact_calls as f and inner join sample_dim as s on f.sample_id = s.sample_id.
Group the joined rows by s.condition.
Use COUNT(*) to count joined call rows for each condition.
Use AVG(s.expr_pgr5) to compute mean expr_pgr5 value across joined rows, then round to 3 decimals.
Return the columns in this exact order: condition, call_count, avg_expr_pgr5.
Sort by condition ascending.

Discovery

0/9 exact. All models returned the right number of condition rows, but every model missed the values. The recurring error was again sample-level aggregation instead of fact-row aggregation; several also leaked the zero-call S5 high-light sample into the average.

Pass 1 - Easy SQL | Prompt 8

Code

```
SELECT
    s.condition,
    COUNT(*) AS call_count,
    ROUND(AVG(CAST(s.expr_pgr5 AS
    DECIMAL(18, 6))), 3) AS avg_expr_pgr5
FROM fact_calls AS f
INNER JOIN sample_dim AS s
  ON f.sample_id = s.sample_id
GROUP BY s.condition
ORDER BY s.condition ASC;
```

Prompt

Inner join fact_calls to sample_dim.
For each condition, compute:
- call_count
- avg_expr_pgr5

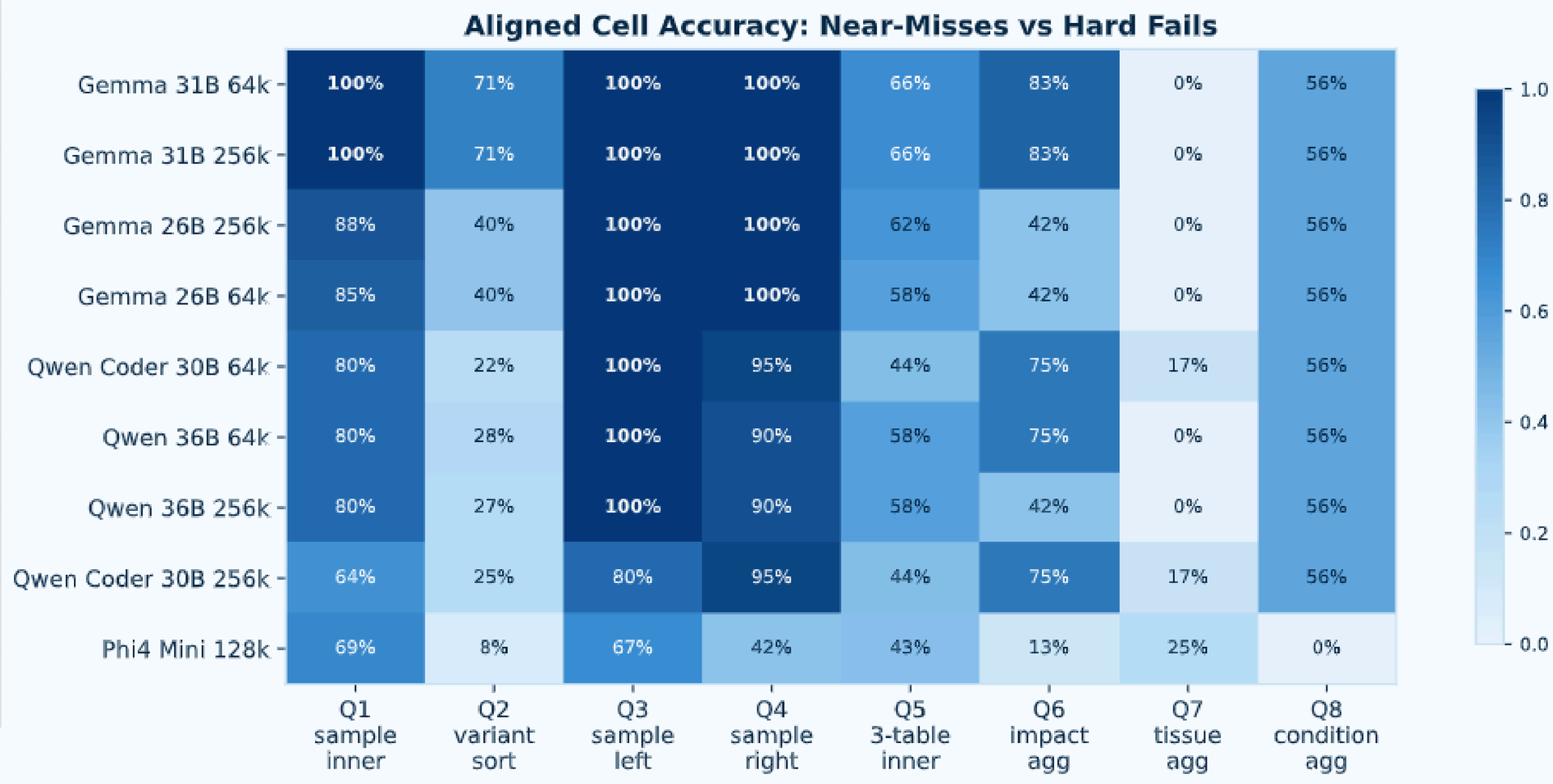
Return: condition, call_count, avg_expr_pgr5
Sort by condition ascending.
Round avg_expr_pgr5 to 3 decimals.

Result

condition	call_count	avg_expr_pgr5
control	4	475.000
drought	2	700.000
high_light	2	800.000

Alligned Cell Accuracy

Model	Average score
Gemma 31B 64k	72.00%
Gemma 31B 256k	72.00%
Qwen Coder 30B 64k	61.13%
Gemma 26B 256k	61.00%
Qwen 36B 64k	60.88%
Gemma 26B 64k	60.13%
Qwen Coder 30B 256k	57.00%
Qwen 36B 256k	56.63%
Phi4 Mini 128k	33.38%



The heatmap compares how well local AI models performed on eight benchmark questions, using aligned cell accuracy as the metric. Gemma 31B variants are the strongest overall performers, with very high accuracy on most questions and perfect scores on several of them. Q7, the tissue aggregation task, is particularly challenging, with several models scoring 0%, suggesting that aggregation and grouped reasoning are much more difficult than joins.

Summary of pass 1

Overview

- Pass 1 did not fail on formatting. All nine models returned valid JSON and usually the expected columns, so the core problem was relational exactness rather than output structure.
- The best models were Gemma 31B 64k and Gemma 31B 256k at 3 exact answers out of 8, or 37.5 percent accuracy. Even the leaders were wrong more often than right.
- The benchmark exposed a clear gap between plausible answers and exact answers. Several models had moderate aligned cell accuracy but still missed exact table equality.
- The main conclusion is simple. Current local models can present structured results, but most are still brittle when they must execute SQL style semantics exactly.

Conclusion

- Pass 1 proves that easy looking SQL prompts are still difficult for local models when the scoring requires exact table equality.
- The best current option in this run was Gemma 31B, but even that family only reached 37.5 percent exact accuracy. That is useful, but not reliable enough to treat as deterministic analytics.
- The benchmark is doing its job well because it separates formatting success from analytical truth. A clean JSON table is not the same as a correct answer.
- The bottom line is that local models are promising assistants for structured work, but they still need verification whenever joins, chained filters, and weighted aggregates matter.

Discovery

Pass one exposes a specific weakness: these models can format structured answers, but they are brittle at exact relational execution. The largest Gemma 31B variants were best, but even they only reached 37.5% exact accuracy. The failures are concentrated around SQL semantics: inner-vs-left filtering, chained joins, sort stability, type fidelity, and fact-row-weighted aggregation.

Thank You